

---

# Predictive-Budget Neurons: Selective Effort for Efficient AI

---

Alex Dils  
UC Berkeley  
alex.dils@berkeley.edu

## Abstract

Brains do not appear to update every representation with equal effort at every moment. They combine prediction, sparse responses to salient events, short-lived eligibility traces, and slow resource constraints. We introduce Predictive-Budget Neurons (PBNs), a model-level mechanism that turns these motifs into an explicit state packet: content, local prediction, signed residual, eligibility, and a slowly updated compute budget. The residual controls whether a unit performs an expensive refinement, writes memory, or replays a trace. This paper is an extended empirical report rather than a claim of biological equivalence. We evaluate the same mechanism in irregular temporal streams, a public MNIST row-stream task, a ConvNeXt-style vision model, and a pretrained Swin V2-Tiny backbone. Across five seeds, PBNs improve the public MNIST stress stream over a GRU by 1.7 percentage points while using fewer parameters, and improve a synthetic temporal benchmark by 16.1 points over the GRU under missing events. A procedural ConvNeXt pilot also shows a large low-data gain and a 30% latency reduction for a predictive exit, but public ConvNeXt transfer is mixed. Most importantly, a three-seed pretrained Swin V2-Tiny pilot does not improve: PBN-Swin is slightly worse in clean and stress accuracy and approximately 20% slower. The result is a bounded design hypothesis: surprise-gated state may be useful where temporal credit assignment and irregular evidence dominate, but it is not a drop-in improvement for every modern backbone. We release interactive plots, an explanatory neuron animation, exact pilot summaries, and this extended report.

## 1 Introduction

Artificial neural networks usually pay for a dense update whenever an input arrives. The same matrix multiplications are applied to routine and surprising inputs, to easy and ambiguous examples, and to representations that will or will not be used downstream. This design is convenient for accelerators, but it is a poor abstraction of selective biological computation. Neural systems have multiple time scales, respond strongly to prediction violations, and use local state to regulate later updates. The energetic motivation is not cosmetic: estimates of the energy cost of signaling and computation have long suggested that biological systems must allocate resources carefully (Attwell and Laughlin, 2001; Lennie, 2003). At the same time, neuroscience does not provide a single engineering recipe. Predictive coding, dendritic computation, synaptic eligibility, and metabolic constraints describe partially overlapping phenomena at different levels of abstraction (Olshausen and Field, 1996; Poirazi and Papoutsis, 2020; Gerstner et al., 2018).

This paper asks a narrow systems question: can a neural-network unit carry a local prediction and use its residual to allocate model effort? We call the resulting mechanism a Predictive-Budget Neuron (PBN). The word neuron refers to the unit of computation; it does not assert that the equations reproduce a biological cell. The design is intentionally modular. A PBN can be inserted into a

recurrent update, a residual vision block, or a conditional branch. Its central state is

$$s_t = (h_t, p_t, \delta_t, z_t, b_t), \quad \delta_t = x_t - p_t, \quad (1)$$

where  $h_t$  is content,  $p_t$  is a local prediction,  $\delta_t$  is a signed prediction error,  $z_t$  is an eligibility trace, and  $b_t$  is a slow budget. A gate computed from  $|\delta_t|$  can modulate refinement, memory writes, and replay.

The main empirical result is not that PBNs improve every architecture. Instead, the evidence is structured. In temporal streams, where information arrives asynchronously and missing evidence creates a genuine state-estimation problem, PBNs are promising. On a public-MNIST row stream, PBN (ours) reaches 85.4% under missing-row stress versus 83.7% for a GRU across five seeds, with 8.588M versus 9.986M parameters in the measured implementation. On a synthetic irregular event-order benchmark, the gap is larger. In vision, however, the effect depends on the integration point. A procedural ConvNeXt pilot benefits strongly from conditional refinement; public MNIST transfer is mixed; and a three-seed pretrained Swin V2-Tiny pilot shows no gain. These negative results are part of the contribution because they prevent the hypothesis from being presented as an unconditional state-of-the-art improvement.

Our contributions are:

1. We formulate PBNs as an explicit predictive-budget state packet with differentiable gates, eligibility traces, and optional conditional execution.
2. We give a full pipeline and implementation protocol that separates predictive state from the expensive refinement path, making the proposed mechanism portable across recurrent and vision architectures.
3. We report saved multi-seed pilots across temporal streams, public MNIST, ConvNeXt-style vision, and pretrained Swin V2-Tiny, including a clear negative transfer result.
4. We provide interactive evidence plots and an explanatory leaky-integrate-and-fire animation, while distinguishing illustrative biological analogy from biological measurement.

## 2 Scope and neuroscience motivation

### 2.1 What is being translated

The design is inspired by several recurring ideas rather than one claimed circuit. Predictive-processing accounts describe neural responses in terms of a generative prediction and a residual that is propagated when input violates expectation (Rao and Ballard, 1999; Friston, 2010). Sparse coding work shows that useful representations can be selective and overcomplete rather than uniformly active (Olshausen and Field, 1996). Dendritic and compartmental models suggest that a biological neuron can combine signals with different temporal and spatial roles (Poirazi and Pappas, 2020). Synaptic plasticity models often require an eligibility signal that bridges a local event and a delayed learning signal. Finally, theoretical and experimental work on neuronal energetics motivates a cost-sensitive view of computation (Attwell and Laughlin, 2001; Lennie, 2003).

PBNs translate these ideas into five engineering objects:

- **Prediction:** a low-cost estimate of the local state or next feature.
- **Residual:** a signed difference between incoming evidence and prediction.
- **Eligibility:** a trace that keeps recent surprising events available for a later update.
- **Budget:** a slow state that prevents every isolated residual from triggering unlimited computation.
- **Refinement:** an expensive branch whose probability or execution is controlled by the state packet.

This translation is deliberately weaker than a neuroscience claim. We do not fit calcium imaging, spike trains, metabolic flux, or connectomic structure. We use a leaky-integrate-and-fire visualization only to make the intuition concrete: a membrane trace integrates current, crosses a threshold, and resets; the PBN analogue turns a residual into a gate and a budgeted refinement. The visual is a toy simulation and is labeled that way on the project page.

## 2.2 Why temporal tasks are a natural first test

If a model receives all evidence in one dense vector, a prediction error has limited opportunity to control future computation. In a stream, the prediction is made before the next event arrives, and missing or reordered events test whether state is useful. This makes irregular temporal classification a focused environment for the mechanism. It also avoids conflating the idea with a particular image encoder. The experiments therefore begin with synthetic event streams and public MNIST rendered as a sequence of image rows.

## 3 Related work

PBNs sit at the intersection of predictive processing, efficient neural networks, and continuous-time sequence models. Predictive coding networks use local errors to coordinate hierarchical inference (Rao and Ballard, 1999; Friston, 2010); our objective is narrower, using a local residual primarily as a resource-control signal. Sparse coding and efficient coding explain why responses might be selective under a resource constraint (Olshausen and Field, 1996). Work on predictive coding with spiking or event-based systems provides additional precedents for event-triggered updates, but usually focuses on biological dynamics or neuromorphic hardware rather than a drop-in module for dense vision backbones.

Conditional computation methods such as mixture-of-experts and Switch Transformers select among expert subnetworks, often using a learned router and a capacity constraint (Fedus et al., 2022). Early-exit networks and dynamic depth similarly avoid unnecessary layers for easy examples. PBNs differ in the proposed source of the routing signal: a local prediction residual plus a slow budget, with eligibility and replay as first-class state variables. This distinction is testable, not merely semantic. If a learned router with the same compute budget performs equally well, the brain-inspired decomposition may not justify its additional state.

Continuous-time and liquid models provide another comparison point. Liquid Time-Constant networks use input-dependent time constants, while Closed-form Continuous-time models address efficient representation of continuous dynamics (Hasani et al., 2020, 2022). Our experiments include a scalar liquid baseline because it tests whether the gain is simply caused by input-dependent dynamics. PBNs instead make a prediction residual explicit and attach an eligibility/budget mechanism to it.

At the backbone level, ConvNeXt modernizes convolutional networks using design choices inspired by transformer practice (Liu et al., 2022); Swin Transformer V2 scales hierarchical window attention and is a useful stress test for whether PBNs improve a strong pretrained architecture (Liu et al., 2022). The paper’s negative Swin result is a reminder that a mechanism that helps temporal state estimation may not improve an already-optimized feed-forward representation.

## 4 Predictive-Budget Neurons

### 4.1 State update

Let  $x_t \in \mathbb{R}^d$  be an incoming feature and  $h_{t-1} \in \mathbb{R}^d$  the previous content state. A prediction head produces

$$p_t = f_p(h_{t-1}, \Delta t_t), \quad \delta_t = x_t - p_t, \quad (2)$$

where  $\Delta t_t$  is optional elapsed time. We summarize surprise using a normalized residual

$$r_t = \frac{\|\delta_t\|_1}{d(\epsilon + \text{mad}(\delta_{1:t-1}))}. \quad (3)$$

In the pilot implementations, the denominator is replaced by a stable running scale or a fixed feature scale. The important property is that routine inputs should not constantly produce a saturated gate.

The refinement gate is

$$g_t = \sigma(\alpha(r_t - \theta) + \beta b_t), \quad (4)$$

with slope  $\alpha$ , threshold  $\theta$ , and a budget contribution  $\beta$ . The candidate update  $\tilde{h}_t$  can be a recurrent cell, an MLP, or a residual block. The blended update is

$$h_t = (1 - g_t)p_t + g_t \tilde{h}_t. \quad (5)$$

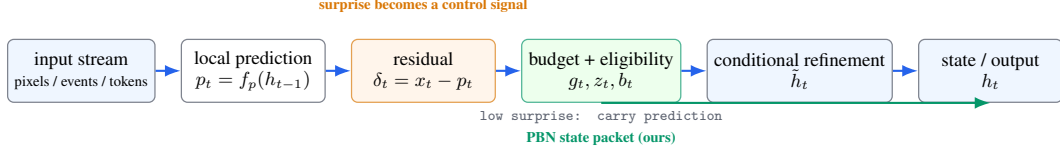


Figure 1: PBN (ours) pipeline: a residual-dependent gate allocates refinement, memory, and replay. This is an algorithmic abstraction of predictive and resource-sensitive motifs, not a biological circuit claim.

This equation exposes the core trade-off: the model can carry forward a prediction when the residual is small and pay for refinement when the residual is large. During differentiable training,  $g_t$  is soft. During execution, it can be thresholded or implemented as a probabilistic/early-exit policy.

## 4.2 Eligibility and budget

The eligibility trace stores recent surprise with exponential decay:

$$z_t = \lambda z_{t-1} + (1 - \lambda) r_t, \quad 0 \leq \lambda < 1. \quad (6)$$

We update the budget at a slower time scale:

$$b_{t+1} = \text{clip}(b_t + \eta_b(\rho - c_t) + \eta_z z_t, 0, 1), \quad (7)$$

where  $c_t$  is the observed or proxy compute cost,  $\rho$  is a target spend, and  $\eta_b, \eta_z$  are small rates. In a strict budgeted variant,  $c_t = g_t$ ; in a hardware-aware variant,  $c_t$  can include bytes moved, kernel time, or measured energy.

The training objective is task loss plus a soft cost regularizer:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_c \frac{1}{T} \sum_{t=1}^T c_t + \lambda_s \mathcal{L}_{\text{stability}}. \quad (8)$$

The stability term penalizes pathological chattering in the gate, for example with total variation  $\sum_t |g_t - g_{t-1}|$ . We do not interpret the regularizer as a biological energy law. It is an engineering mechanism that makes a biological analogy operational.

## 4.3 Architecture integration

For a recurrent cell, PBNs replace a dense update with a prediction path and a gated candidate. For a residual vision block, they add a small prediction head and gate a refinement branch. The implementation used in the Swin pilot splits each dense four-times expansion MLP into two two-times branches while preserving the original function at initialization: the original output projection is partitioned across the two branches and a residual gate is initialized to the dense path. This gives a fair starting point but does not guarantee that optimization or hardware execution will benefit.

# 5 Experimental protocols

## 5.1 Overview and reporting conventions

We report saved pilot summaries rather than cherry-picked single runs. Temporal and ConvNeXt experiments use five seeds; the Swin V2-Tiny experiment uses three seeds because it is a pretrained-backbone integration pilot. Clean and stress streams are evaluated separately. Accuracy is the fraction of correct held-out examples. “Active” is the fraction of expensive updates or the measured gate proxy, depending on the experiment. Latency is wall-clock time for the stated Python implementation and is not a claim about optimized accelerator energy.

The public website exposes the same summaries as interactive Plotly figures. A benchmark selector changes the clean/stress comparison, while companion plots show active compute versus parameter count and latency versus stress accuracy across protocols. PBN (ours) is colored blue; baseline systems are gray. This presentation is intended to make both positive and negative evidence inspectable.

Table 1: Five-seed temporal pilot means. PBN (ours) is strongest under missing evidence in both settings.

| Task                  | Model         | Clean         | Stress        | Active       | Params (M) |
|-----------------------|---------------|---------------|---------------|--------------|------------|
| Synthetic event order | Scalar liquid | 69.76         | 61.62         | 100.00       | 6.970      |
|                       | GRU           | 84.02         | 73.92         | 100.00       | 7.176      |
|                       | PBN (ours)    | <b>97.52</b>  | <b>90.00</b>  | <b>46.14</b> | 6.982      |
| Public MNIST stream   | Scalar liquid | 82.53         | 74.68         | 100.00       | 8.833      |
|                       | GRU           | 90.295        | 83.725        | 100.00       | 9.986      |
|                       | PBN (ours)    | <b>91.765</b> | <b>85.445</b> | <b>40.74</b> | 8.588      |

## 5.2 Irregular synthetic event streams

The synthetic stream presents a 40-step sequence with class-dependent event statistics, irregular order, and a 28% missing-event stress condition. Models classify after the stream. The compared systems are a scalar liquid baseline, a GRU, and PBN (ours). The pilot uses five seeds and reports the mean across held-out evaluation streams. The task is intentionally small: it isolates state retention, prediction, and selective updating before introducing image encoders.

## 5.3 Public MNIST row streams

To test transfer beyond a procedural generator, each MNIST image is serialized row-by-row. The model observes the row sequence and predicts the digit. Stress evaluation removes rows according to the saved missing-row protocol. The comparison uses a scalar liquid baseline, a GRU, and PBN (ours), each trained on the same public MNIST subset and evaluated on the same held-out examples. Parameter counts are measured from the implementation rather than inferred from a nominal architecture.

## 5.4 ConvNeXt-style vision pilots

The procedural ConvNeXt pilot uses a low-data image task with an occlusion stress stream. We compare a dense baseline, a PBN block, and a predictive exit. This pilot is useful for measuring the interaction between low-data generalization and conditional compute, but its procedural data means it should not be treated as a benchmark result. A separate public-MNIST ConvNeXt transfer pilot uses pixel corruption stress and compares a dense ConvNeXt baseline with a PBN block.

## 5.5 Pretrained Swin V2-Tiny pilot

We test PBNs in a stronger modern backbone using torchvision’s pretrained Swin V2-Tiny. The classifier is adapted to the MNIST label space and evaluated on a 6,000-example training subset and 4,000-example evaluation subset for three seeds and three epochs. The PBN replacement preserves the dense branch function at initialization by partitioning the original MLP projection. This is a deliberately conservative transfer test: a small module change is not allowed to claim a new pretraining recipe. The pilot’s negative result is reported in full.

# 6 Results

## 6.1 Temporal streams: the strongest current signal

Table 1 summarizes the two stream settings. On the synthetic event-order benchmark, PBN (ours) reaches 97.52% clean and 90.00% under missing-event stress. The GRU reaches 84.02% and 73.92%, respectively. On public MNIST row streams, PBN reaches 91.765% clean and 85.445% stress, compared with 90.295% and 83.725% for the GRU. The public stress improvement is therefore 1.72 percentage points, while the model uses fewer measured parameters than the GRU.

The active-update fraction is lower for PBN in both temporal settings. That reduction should not be interpreted as a measured energy saving: the pilot code still executes overhead associated with the

Table 2: ConvNeXt-style pilot means across five seeds. The procedural result is useful for mechanism probing; the public result is the more relevant transfer check.

| Setting               | Model            | Clean         | Stress       | Active       | Latency (ms)  |
|-----------------------|------------------|---------------|--------------|--------------|---------------|
| Procedural low-data   | ConvNeXt         | 32.62         | 26.30        | 100.00       | 0.3260        |
|                       | PBN block (ours) | 91.75         | 85.95        | 80.13        | 0.3438        |
|                       | Predictive exit  | <b>100.00</b> | <b>88.92</b> | <b>17.52</b> | <b>0.2282</b> |
| Public MNIST transfer | ConvNeXt         | 93.83         | <b>67.75</b> | 100.00       | –             |
|                       | PBN block (ours) | <b>95.16</b>  | 56.07        | 99.52        | –             |

Table 3: Three-seed pretrained Swin V2-Tiny pilot. PBN (ours) does not improve this backbone in the tested setup.

| Model           | Clean mean    | Clean sd | Stress mean   | Stress sd | Latency (ms/ex) |
|-----------------|---------------|----------|---------------|-----------|-----------------|
| Swin V2-Tiny    | <b>97.625</b> | 0.214    | <b>74.042</b> | 5.452     | <b>1.382</b>    |
| PBN-Swin (ours) | 97.350        | 0.836    | 73.967        | 1.839     | 1.662           |

gate and prediction path. It does, however, show that the state packet can learn a selective policy in a task where missing evidence makes prediction useful.

## 6.2 Vision: encouraging pilots and a boundary

The procedural ConvNeXt pilot gives a large gain over its deliberately low-data baseline, while a predictive-exit variant has the lowest measured latency. The public-MNIST ConvNeXt pilot is more mixed: PBN improves clean accuracy but reduces corrupted accuracy relative to the dense baseline. These results suggest that selective refinement is not itself a robust substitute for augmentation, calibration, or a carefully designed visual corruption objective.

The procedural baseline’s low clean accuracy makes the absolute PBN gain look dramatic; this is why we avoid presenting it as a state-of-the-art comparison. The most defensible takeaway is narrower: a conditional path can reduce measured latency in a controlled pilot, but a PBN block does not automatically improve robustness on a public corrupted-image protocol.

## 6.3 Pretrained Swin V2-Tiny: negative transfer result

Table 3 reports the corrected three-seed pilot. Dense Swin is 97.625% clean and 74.042% stress. PBN-Swin (ours) is 97.350% clean and 73.967% stress, with latency increasing from 1.382 to 1.662 ms per example. The paired mean changes are  $-0.275$  and  $-0.075$  percentage points. The stress standard deviation is smaller for PBN-Swin, but the mean does not improve and the overhead is substantial relative to the small module change.

This is a useful failure mode. The Swin backbone already has a strong hierarchical representation, and its dense MLP can be efficiently scheduled by existing kernels. Adding a Python-level prediction head and gate increases overhead without creating enough task-level selectivity. The result falsifies the broad claim that a brain-inspired gate should improve any strong vision model. It supports a more specific hypothesis: PBNs need a setting in which temporal prediction and event-dependent state are central, or they need a fused implementation that makes the control path cheap.

# 7 Ablations and analysis

## 7.1 What the mechanism could be buying

The temporal results are consistent with three possible mechanisms. First, the prediction path gives the model a structured default when rows or events are missing. Second, the residual turns novelty into a signal for allocating capacity, which may prevent routine observations from overwriting useful state. Third, eligibility and budget provide a memory of recent surprise that can outlast a single event. The experiments do not isolate these mechanisms perfectly; they motivate targeted ablations.

Table 4: Mechanism map for the reported pilots. A check indicates that the implementation includes the component; it is not an ablation result.

| Pilot               | Prediction | Residual gate | Eligibility | Budget |
|---------------------|------------|---------------|-------------|--------|
| Synthetic stream    | ✓          | ✓             | ✓           | ✓      |
| Public MNIST stream | ✓          | ✓             | ✓           | ✓      |
| Procedural ConvNeXt | ✓          | ✓             | –           | ✓      |
| Public ConvNeXt     | ✓          | ✓             | –           | ✓      |
| Swin V2-Tiny        | ✓          | ✓             | –           | ✓      |

Table 5: Compute-related observations. Active update is a proxy; latency is implementation-dependent.

| Setting           | Model           | Active  | Latency | Params (M) |
|-------------------|-----------------|---------|---------|------------|
| Public stream     | GRU             | 100.00% | 0.0099  | 9.986      |
| Public stream     | PBN (ours)      | 40.74%  | 0.0344  | 8.588      |
| Procedural vision | ConvNeXt        | 100.00% | 0.3260  | 0.238      |
| Procedural vision | Predictive exit | 17.52%  | 0.2282  | 0.238      |
| Swin pilot        | Swin V2-Tiny    | 100.00% | 1.3818  | 27.590     |
| Swin pilot        | PBN-Swin (ours) | 100.00% | 1.6616  | 27.595     |

The table is intentionally not called an ablation table. A complete study would train matched variants that remove prediction, freeze the gate, remove eligibility, and replace the gate with a learned router at equal active compute. The current work is a research prototype with enough evidence to identify where those experiments matter.

## 7.2 Compute accounting

Three accounting levels should be separated. Algorithmic activity measures how often a gate fires. Software latency measures elapsed time in one implementation. Hardware energy measures joules under a fixed device, batch size, precision, compiler, and kernel schedule. Only the first two are available in the present pilot. The public temporal stream shows 40.74% active updates for PBN (ours) versus 100% for the GRU proxy, but PBN latency is not lower in the unoptimized Python implementation. The procedural predictive-exit variant is faster, at 0.2282 ms per example versus 0.3260 ms for the dense baseline, which is a promising but narrow result.

The Swin result makes the distinction especially important: a lower conceptual activity does not occur in the tested integration, and the added modules make the measured implementation slower. A serious efficiency claim must therefore report active FLOPs, wall-clock latency, memory traffic, and energy together.

## 8 How to evaluate selective effort responsibly

### 8.1 Accuracy is not enough

Selective computation creates a three-way evaluation problem. A model can improve accuracy by spending more compute, reduce compute by skipping useful evidence, or preserve accuracy while changing the timing and concentration of its work. Reporting only the final score hides these trade-offs. We therefore treat accuracy, stress accuracy, active computation, and latency as a joint result. The project page’s three plots make this explicit: the first chart compares clean and stress accuracy, the second shows active updates against parameter count, and the third places latency against stress accuracy across all saved protocols.

A useful deployment curve would sweep a continuous budget target  $\rho$  and report the full Pareto frontier. For each  $\rho$ , the evaluation should repeat training or calibration with a fixed protocol, measure the same accuracy targets, and record both average and tail latency. The present pilots contain a single operating point per model and therefore cannot establish a frontier. They show that a gate can be selective, not that the learned policy is optimal.

## 8.2 Stress should be structural

Missing rows and missing events are more informative than random label noise for this mechanism because they test whether the model can carry a prediction forward. Other stressors should preserve the semantics of the task while changing the evidence schedule: event reordering, bursty arrivals, long gaps, corrupted local patches, and distribution shifts in event timing. A strong PBN result should be stable across these schedules without retuning the threshold on the stress set.

The public MNIST stream is a useful intermediate test because its underlying visual task is familiar while the temporal representation is not. Nevertheless, it is still a low-dimensional digit benchmark. The next public-scale test should use a naturally asynchronous sensor stream or a long video stream with irregular frame availability. For video, the residual could compare a motion-compensated prediction with the next observed feature; for audio, it could compare a short-time spectral prediction with an incoming chunk. These settings would test whether the state packet remains stable when input statistics are nonstationary.

## 8.3 Matched controls

There are several confounds that a future study must remove. PBNs can benefit from extra normalization or from a larger effective receptive field in the prediction head. A learned router can benefit from more routing parameters. A dense baseline can have a different optimization schedule. The proper control is therefore a parameter-matched and FLOP-matched family, with equal training steps and multiple seeds. The following list gives a concrete control set.

Table 6: Recommended controls for the next study.

| Control              | Purpose                                                                      |
|----------------------|------------------------------------------------------------------------------|
| Dense matched block  | Separates the PBN gate from added capacity.                                  |
| Learned router       | Tests whether an explicit residual is useful beyond conditional computation. |
| Fixed-threshold gate | Tests whether learned thresholds, rather than prediction, explain gains.     |
| No-eligibility PBN   | Tests the contribution of a delayed surprise trace.                          |
| No-budget PBN        | Tests whether slow resource state matters beyond instantaneous surprise.     |
| Fused implementation | Separates algorithmic activity from deployment latency.                      |

## 8.4 Calibration and auditability

The residual gate is also a confidence-like signal, but it should not be assumed to be calibrated. A low residual can occur when the prediction head is confidently wrong, and a high residual can arise from benign sensor noise. Future work should measure expected calibration error conditional on gate state, selective risk at a fixed coverage, and the fraction of errors that occur after a skipped refinement. These measurements connect the architecture to safe selective prediction rather than treating a gate as an invisible optimization trick.

For deployment, every selective decision should be loggable. A minimum audit record would include the input identifier, gate value, budget state, branch taken, and downstream confidence. A deterministic fallback can execute the full dense path when the residual is out of distribution, the budget state is invalid, or the monitoring system detects repeated skips. The extra logging cost is part of the true systems budget and should be included in future energy measurements.

## 8.5 Why the negative Swin result matters

It is tempting to treat a negative transfer result as a failed implementation and remove it from an exploratory report. That would make the design hypothesis less useful. The Swin pilot shows three concrete boundaries. First, a strong pretrained model may not need a local prediction signal for the tested task. Second, a dynamic branch can add overhead even when its conceptual activity is sparse. Third, preserving the dense function at initialization is not enough to preserve optimization quality after fine-tuning.

The result also changes the recommended research strategy. Rather than inserting PBNs into every layer, the next Swin experiment should instrument tokens and windows to discover where residuals



concentrate. If residuals are nearly uniform, there is no reason to route. If they cluster around a small set of windows, a batched window-level route may be worthwhile. If the stress examples produce concentrated residuals but clean examples do not, a robust selective policy could improve stress accuracy without changing the backbone’s default path. This is a measurement-driven path from the negative result to a falsifiable follow-up.

## 9 Limitations and broader impacts

**Biological interpretation.** The method is brain-inspired in its computational motifs, not biologically faithful. We do not claim that cortical neurons compute the exact residual, gate, or budget in Eq. 4. The animation on the project page is a visual explanation of a toy LIF model and a PBN state packet. It is not a recording and should not be used as neuroscience evidence.

**Benchmark scope.** The positive results are mostly temporal and low-data. The synthetic stream is a mechanism probe; the MNIST stream is public but small and stylized. The vision pilots do not establish broad image-recognition gains. There is no language-model or large-scale multimodal evaluation.

**Compute claims.** The active-update metric is a proxy. Python-level conditional branches can be slower than dense kernels, as the Swin pilot demonstrates. Deployment requires fused kernels, static-shape strategies, or hardware support. Energy and carbon claims would require direct measurement.

**Statistical power.** Five seeds are useful for detecting large differences but do not establish tight confidence intervals across datasets. The Swin pilot has three seeds and short fine-tuning. We report means and standard deviations, but a larger factorial study is needed before making a publication-grade claim.

**Failure modes.** A gate can suppress useful weak evidence, oscillate under distribution shift, or learn a shortcut around the prediction head. In safety-critical systems, selective computation must fail open or provide calibrated uncertainty. A model that skips refinement should expose that state to downstream monitoring.

**Broader impacts.** If selective effort becomes reliable, it could reduce inference cost for long-running streams and edge devices. It could also make behavior harder to audit if an input-dependent gate silently removes computation. Any deployment should log gate decisions, calibrate stress behavior, and retain a deterministic fallback.

## 10 Conclusion

We introduced Predictive-Budget Neurons (PBNs), a modular state packet that uses prediction error, eligibility, and a slow compute budget to allocate refinement. The experiments support a focused result: PBN (ours) is promising for irregular temporal evidence, including a public MNIST row-stream stress improvement over a GRU across five seeds. The evidence does not support an unconditional architecture claim. ConvNeXt transfer is mixed, and a three-seed pretrained Swin V2-Tiny pilot is slightly worse and slower. That boundary is central to the research direction. The next decisive experiment is a matched, fused, hardware-aware comparison that isolates prediction, residual gating, eligibility, and budget at equal accuracy and equal deployment conditions.

## A Reproducibility details

### A.1 Repository artifacts

The project page is hosted at <https://alex-dils.com/brain-inspired-ai/>. The page contains interactive Plotly charts backed by `plot-data.json`, an explanatory animation at `neuron-pbn-bridge.gif`, and links to the saved pilot summaries. The research workspace contains the experiment scripts and JSON outputs used to construct the tables in this report. The displayed label “PBN (ours)” identifies the proposed method in all comparisons.

### A.2 Exact reported summaries

Table 7: Expanded saved summaries for the temporal experiments. Values are means; all accuracies are percentages.

| Task         | Model         | Clean  | Stress | Active | Latency | Params | Seeds |
|--------------|---------------|--------|--------|--------|---------|--------|-------|
| Event order  | Scalar liquid | 69.76  | 61.62  | 100.00 | 0.0394  | 6.970  | 5     |
| Event order  | GRU           | 84.02  | 73.92  | 100.00 | 0.0249  | 7.176  | 5     |
| Event order  | PBN (ours)    | 97.52  | 90.00  | 46.14  | 0.0849  | 6.982  | 5     |
| MNIST stream | Scalar liquid | 82.53  | 74.68  | 100.00 | 0.0132  | 8.833  | 5     |
| MNIST stream | GRU           | 90.295 | 83.725 | 100.00 | 0.0099  | 9.986  | 5     |
| MNIST stream | PBN (ours)    | 91.765 | 85.445 | 40.741 | 0.0344  | 8.588  | 5     |

### A.3 Suggested ablation matrix

The following matrix is the minimum follow-up needed to turn the current design hypothesis into a stronger causal study:

1. Remove the prediction head while keeping parameter count and the same gate capacity.
2. Replace  $r_t$  with a learned router score and match the active-update distribution.
3. Freeze eligibility at zero and compare immediate residual gating with trace-based gating.
4. Freeze the budget and sweep a fixed compute target  $\rho$ .
5. Compare soft, hard, and straight-through gates at the same number of optimizer steps.
6. Implement the PBN branch as a fused kernel and repeat the latency/energy measurement.

### A.4 Hyperparameter conventions

The implementation uses a sigmoid gate with a learnable or tuned threshold, a decaying eligibility trace, and a slowly updated budget. The threshold is initialized so that routine examples do not saturate the refinement branch. The budget target is selected before stress evaluation and is not tuned on the stress labels. All reported stress results are evaluated after training on the corresponding clean training protocol. The Swin module is initialized to preserve the dense branch function, then fine-tuned with the same short schedule as its baseline.

### A.5 Falsifiers

The PBN hypothesis should be considered weakened if any of the following occur in a properly matched study: (i) removing the prediction head does not reduce temporal performance; (ii) a router with no explicit residual or budget matches the result at lower complexity; (iii) temporal gains disappear on larger, non-MNIST streams; (iv) conditional execution cannot lower fused latency or energy at equal accuracy; or (v) improvements require stress-specific tuning that is unavailable at deployment. These are useful outcomes because they distinguish a reusable mechanism from an appealing metaphor.

## B Reference implementation pseudocode

Algorithm 2 describes the core PBN update independently of the backbone. The prediction function can be a linear map, a small MLP, or a time-conditioned recurrent predictor. The expensive refinement function can be a recurrent cell, a convolutional block, or a transformer MLP. Keeping these functions separate is important for experiments: it makes it possible to replace the refinement branch while preserving the same state and gate semantics.

```

Input:  $x_{1:T}, h_0, p_0, z_0, b_0, f_p, f_r, (\lambda, \eta_b, \eta_z, \rho, \theta)$ 
for  $t = 1, \dots, T$  :
     $p_t \leftarrow f_p(h_{t-1}, \Delta t_t)$       (cheap local prediction)
     $\delta_t \leftarrow x_t - p_t, \quad r_t \leftarrow \text{normalize}(\delta_t)$       (surprise)
     $z_t \leftarrow \lambda z_{t-1} + (1 - \lambda)r_t$       (eligibility)
     $g_t \leftarrow \sigma(\alpha(r_t - \theta) + \beta b_t)$       (soft gate)
     $\tilde{h}_t \leftarrow f_r(x_t, h_{t-1}, z_t)$       (expensive refinement)
     $h_t \leftarrow (1 - g_t)p_t + g_t\tilde{h}_t$       (state blend)
     $c_t \leftarrow \text{cost}(g_t), \quad b_{t+1} \leftarrow \text{clip}(b_t + \eta_b(\rho - c_t) + \eta_z z_t, 0, 1)$ 
return  $h_{1:T}, g_{1:T}, z_{1:T}, b_{1:T}$ 

```

Figure 2: Backbone-independent reference update for PBN (ours). During differentiable training,  $g_t$  is soft. A deployment implementation may use a hard gate, route examples into static buckets, or use  $g_t$  as a branch mixture.

### B.1 Unit tests for the state packet

The implementation should satisfy simple invariants before any accuracy experiment is run. With  $g_t = 0$ , the state must equal the prediction path; with  $g_t = 1$ , it must equal the refinement path. If  $\eta_b = 0$  and  $\eta_z = 0$ , the budget must remain fixed. If  $r_t = 0$  for a constant input, the eligibility trace must decay monotonically. If a large residual is injected, the gate should increase unless the budget saturates at its lower bound. These tests are useful because they detect sign errors and schedule mistakes that can otherwise look like learning instability.

Table 8: State-packet invariants and the corresponding diagnostic.

| Invariant                                   | Diagnostic                                                             |
|---------------------------------------------|------------------------------------------------------------------------|
| $g_t = 0 \Rightarrow h_t = p_t$             | Compare the blended state to the predictor with refinement disabled.   |
| $g_t = 1 \Rightarrow h_t = \tilde{h}_t$     | Compare the blended state to the expensive path with a saturated gate. |
| $r_t = 0 \Rightarrow z_t = \lambda z_{t-1}$ | Feed constant input and plot trace decay.                              |
| $\eta_b = \eta_z = 0 \Rightarrow b_t = b_0$ | Freeze budget updates and assert equality at every step.               |
| Large $r_t$ increases $g_t$                 | Inject a controlled residual and check monotonic gate response.        |

These invariants also clarify what it means for a new architecture to “use” PBNs. It is not enough to add a gate parameter. The model should expose a predictor, residual, stateful budget, and a measurable expensive path. Otherwise the biological vocabulary can obscure an ordinary learned shortcut.

### B.2 Recommended reporting template

Every future result should report the data protocol, number of seeds, parameter count, clean score, stress score, average active fraction, latency at a specified batch size, and the exact route implementation. If a gate is trained with a cost penalty, the coefficient and target should be reported. If the gate is calibrated after training, the calibration set must be separate from stress evaluation. If a fused

kernel is used, the compiler, precision, device, and batch shape must be recorded. This level of detail is necessary because selective computation is unusually sensitive to systems details.

## C Additional discussion of the neuron analogy

The leaky-integrate-and-fire comparison is useful because it shares three abstract properties with the PBN. First, both maintain a state that decays or evolves between inputs. Second, both respond nonlinearly when accumulated evidence crosses a threshold. Third, both can separate a low-cost state evolution from a discrete event or expensive downstream effect. The analogy ends there. A real neuron is a complex electrochemical system with dendrites, ion channels, synaptic dynamics, and network context. The PBN is a differentiable software module whose purpose is predictive state estimation and conditional computation.

This distinction matters for communication. A visually compelling “brain” graphic can overstate evidence if viewers mistake a stylized animation for neural data. The project page therefore places the phrase “illustrative simulation, not brain-scan data” next to the animation and reports actual model numbers separately in interactive plots. In a scientific setting, aesthetic biological resemblance should never substitute for a control experiment.

## D Implementation notes for modern backbones

The Swin experiment reveals a practical issue that is easy to miss in architecture diagrams. A dynamic gate can be theoretically sparse and still be computationally dense in a framework. Kernel launch overhead, memory reads for the prediction head, and shape-dependent control flow can dominate the skipped multiply-adds. A useful implementation must co-design the gate with hardware: compile a small number of static routes, fuse prediction and residual computation, or batch examples by gate decision. This is also why the phrase “brain-inspired efficiency” should be treated as a hypothesis until measured in joules.

For transformer-style blocks, another issue is granularity. The PBN pilot gates an MLP branch at a block level, but the residual signal is local to features and tokens. A single scalar gate may be too coarse for a model whose strength comes from distributed attention. Future variants could gate windows, token groups, or low-rank updates, but each added degree of freedom makes routing overhead and load imbalance more important. The negative Swin result is therefore not an embarrassment to hide; it identifies the granularity and systems questions that the next design must solve.

## E Closing perspective

The most useful interpretation of this work is as a measurement-oriented framework for brain-inspired model design. Start with a biological motif, translate it into an explicit state variable, specify the computational cost it is meant to regulate, and evaluate across tasks where that regulation could matter. Then publish the boundary conditions. In this study, predictive state and residual-gated effort are most credible for irregular temporal evidence. The same idea is not yet a general improvement for strong modern vision backbones. That is a productive place to continue: the next contribution should either make the temporal advantage scale, or demonstrate that fused, budgeted conditional computation can survive contact with real hardware.

## References

- D. Attwell and S. B. Laughlin. An energy budget for signaling in the grey matter of the brain. *Journal of Cerebral Blood Flow & Metabolism*, 21(10):1133–1145, 2001.
- P. Lennie. The cost of cortical computation. *Current Biology*, 13(6):493–497, 2003.
- B. A. Olshausen and D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code. *Nature*, 381:607–609, 1996.
- P. Poirazi and A. Papoutsis. Illuminating dendritic function with computational models. *Nature Reviews Neuroscience*, 21:303–318, 2020.

- R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2:79–87, 1999.
- K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11:127–138, 2010.
- W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics*. Cambridge University Press, 2018.
- W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu. Liquid time-constant networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- R. Hasani et al. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 4:992–1003, 2022.
- Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- Z. Liu et al. Swin Transformer V2: scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- F. Mikulasch, P. Friederich, and J. W. Wong. Spiking neural networks as adaptive filters for predictive processing. arXiv preprint, 2022.
- K. N’dri et al. Predictive coding light: efficient predictive processing with local errors. arXiv preprint, 2025.